

Week 4: Running the Flutter Apps in VS Code

Dr M AOUE

Guide to the week 4 programs

About this guide. It explains how to run the week 4 Flutter apps in **Visual Studio Code**, and describes what each app should show on the screen. Each file starts with a comment that says what it teaches and what result to expect: run it, then compare your screen with that comment.

1 What you need

- **Flutter** installed, and **Visual Studio Code** with the **Flutter** extension. If something is missing, follow the install guide `001_install_vscode`. Check with `flutter doctor` in a terminal.
- **A device to run on.** This week you do not need an Android emulator: **Chrome** or **Windows (desktop)** is enough. `flutter doctor` shows the devices you have.

2 Get the week 4 code and open it

If you already copied the repository, update it: open a terminal in the repository folder and type

```
git pull
```

Otherwise download it with `git clone https://github.com/maoude/mobile_pro_2026.git`. In VS Code choose **File** → **Open Folder...** and open the project folder `code/week04/flutter_examples`. Open the folder of the project itself, not a parent folder: this is how VS Code recognises a Flutter project.

3 Prepare the project (once)

The folder contains only the Dart code. Open a terminal in it (**Terminal** → **New Terminal**) and type:

```
flutter pub get
flutter create .
```

`flutter pub get` downloads the packages. `flutter create .` adds the platform folders (`android`, `web`, `windows...`) that are needed to run the app. It does not change the existing files. You only do this once.

4 Choose a device

At the bottom right of the VS Code window, click the device name (it may say **No Device**) and choose **Chrome**, **Windows (desktop)**, or an Android emulator if you created one in Android Studio's Device Manager.

5 Run an app

Every file in the folder `lib` is a complete app with its own `main`:

1. Open the file, for example `lib/05_text_style.dart`.
2. Click **Run** in the small **Run | Debug** line above `void main()`. The first start takes a little while.
3. The app opens in a window (or in Chrome). The file `lib/main.dart` starts the last step of the lecture (see Section 6).

From a terminal, run any file with the `-t` option: `flutter run -t lib/05_text_style.dart`.

Change the code and see the result at once

While the app runs, change the code and save the file (`Ctrl+S`): Flutter updates the running app in a second. This is called **hot reload**. `Ctrl+Shift+F5` does a **hot restart**, which starts the app again from the beginning. `Shift+F5` stops it.

Run the automated checks

The project also contains tests, in the folder `test`. Each test starts one step of the lecture and checks what appears on the screen. Type `flutter test`: it should end with **All tests passed!**

6 The apps: what you should see

The steps build one application. Run them in order and compare each screen with the previous one.

File	What you see
<code>00_stateless_vs_stateful.dart</code>	The text “Hello, Rami” and a checkbox with “Not accepted”. Tap the checkbox: it is ticked and the text becomes “Accepted” (a stateful widget).
<code>01_hello_container.dart</code>	A completely white screen: the smallest app.
<code>02_hello_text.dart</code>	“Hello World” in the middle, in an ugly red style with a yellow underline. This is expected: the text is not inside a <code>Scaffold</code> yet.
<code>03_scaffold.dart</code>	“Hello World” in a normal small dark text: the <code>Scaffold</code> fixes the style.
<code>04_appbar.dart</code>	A bar at the top with the centered title “Home Page”.
<code>05_text_style.dart</code>	“Hello World” is now larger and deep purple.
<code>06_custom_widget.dart</code>	The words “Text 1”, centered, in the purple style: the text now comes from your own widget <code>MyTextWidget</code> .
<code>07_column.dart</code>	“Text 1” and “Text 2” one below the other, with a space above each.
<code>08_row.dart</code>	The same, plus one line with “Text 3”, “Text 4” and “Text 5” spread evenly. It is also the app that <code>main.dart</code> starts.
<code>09_logical_pixels.dart</code>	Four lines about your screen: its size in logical pixels, the device pixel ratio, its size in physical pixels, and what 16 logical pixels are in physical pixels. The numbers depend on your device.

Going further: three extra apps

These three are not in the slides; they add options that almost every app uses.

File	What you see
10_appbar_options.dart	A teal app bar with a shadow, a menu icon on the left, and search and settings icons on the right. Tap an icon: a message appears at the bottom.
11_scaffold_extras.dart	A page with a light background, a Card in the middle, and a bar at the bottom of the screen.
12_expanded.dart	A grey strip of fixed height, then three colored areas that share the rest of the screen in the ratio 1 : 2 : 1. Resize the window: the ratio stays the same.

Part 2: architecture and widget types

These three go with the document *Week 4, Part 2: Flutter Architecture and Widget Types*. Example 13 shows an image from the folder **assets**. The folder is already declared in the project file `pubspec.yaml`, so there is nothing to set up.

File	What you see
13_widget_types.dart	A scrollable page. First the visible widgets: a bold centered text, three buttons, a small blue logo, a red heart, and the line “Last button: none” that changes when you tap a button. Then the layout widgets: a Row of three labels and a Stack (a red square on a bigger blue square).
14_gestures.dart	A square that counts your touches: “Taps: 0”. Each tap adds 1; a long press (press and hold) resets it to 0.
15_widget_tree_dump.dart	Two lines of text and a round button with a tree icon. Press the button, then look at the terminal (or the Debug Console of VS Code): Flutter prints the widget tree of the app there.

Try it yourself

Change the code of the last step and watch the result after each save:

- change `fontSize` and `color` in `MyTextWidget`;
- add a sixth text to the `Row`;
- replace `spaceEvenly` with `spaceAround` or `spaceBetween` and compare;
- change a `SizedBox` height from 16.0 to 40.0;
- in `12_expanded.dart`, change the `flex` values (for example 1, 1, 3) and add a fourth `Expanded`;
- in `14_gestures.dart`, add an `onDoubleTap` that adds 10, and see how the taps behave;
- in `15_widget_tree_dump.dart`, add a third `Text` to the `Column` and press the button again: find it in the tree.

7 Quick troubleshooting

Problem	What to do
No Run Debug line above main	Install the Flutter extension, and open the project folder <code>flutter_examples</code> itself.
The device list at the bottom right says No Device	Click it and choose Chrome . Check that <code>flutter doctor</code> shows Chrome.
An error says that no platform is configured	Run <code>flutter create .</code> once in the project folder (Section 3).
Step 2 looks ugly	That is the point of the step. Step 3 fixes it with a Scaffold .
Waiting for another <code>flutter</code> command to release the startup lock	Another Flutter command is running, for example in another window. Wait for it, or close it.

This document is maintained by Dr. Mohamad AOUE
Last updated: 2026-09-21